PROGRAMMING & CS FUNDAMENTALS

Searching Algorithms Visual Guide

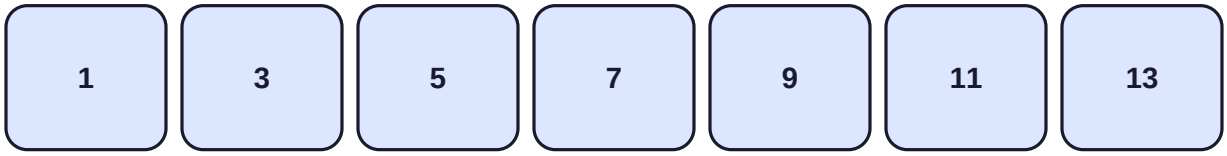
Finding needles —
from linear scans to binary search mastery.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Linear vs Binary: The Setup

Sorted input unlocks logarithmic search.



linear: check 1,3,5,7... until hit -> O(n)

binary: check 7, then half the range -> O(log n)

- Binary search needs RANDOM ACCESS + SORTED order — arrays, not linked lists.
- n=1B: linear touches a billion; binary touches ~30. Sorting once pays forever.
- Unsorted + one search? Linear wins. Unsorted + many searches? Sort first, then binary.

STEP 2/6

Binary Search Mechanics

Halve the range every step — the loop invariant is everything.

lo hi

```
lo, hi = 0, n - 1
while lo <= hi:                      # invariant: answer in [lo, hi]
    mid = lo + (hi - lo) // 2        # no overflow, unlike (lo+hi)//2
    if a[mid] == T: return mid
    elif a[mid] < T: lo = mid + 1    # discard left half
    else: hi = mid - 1              # discard right half
return -1
```

The two classic bugs: mid overflow in fixed-width ints, and infinite loops from lo=mid / hi=mid without +/-1. Exclude mid every step.

STEP 3/6

Bounds Variants: lower_bound / upper_bound

Duplicates? Find the FIRST or LAST occurrence, not just any.

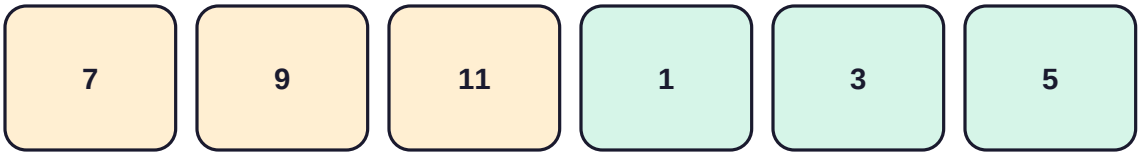
```
# lower_bound: first i with a[i] >= T   (bisect_left)
lo, hi = 0, n
while lo < hi:
    mid = (lo + hi) // 2
    if a[mid] < T: lo = mid + 1
    else: hi = mid          # mid could BE the answer: keep it
return lo
```

- lower_bound(5) -> index 2 (first 5); upper_bound(5) -> index 5 (first > 5).
- Count of T = upper - lower. Insertion point = lower_bound — keeps array sorted.
- Half-open [lo, hi) + 'hi = mid keeps candidate' is the bug-free template.

STEP 4/6

Search in Rotated Sorted Array

One half is always sorted — find it, then decide.



pivot somewhere inside; target T=3

```
# each step: identify the SORTED half via a[lo] <= a[mid]
if a[lo] <= a[mid]:            # left half sorted
    if a[lo] <= T < a[mid]: hi = mid - 1
    else: lo = mid + 1
else:                          # right half sorted
    if a[mid] < T <= a[hi]: lo = mid + 1
    else: hi = mid - 1
```

With duplicates, a[lo]==a[mid] hides which half is sorted — shrink lo+=1 and continue (degrades toward O(n) worst case).

STEP 5/6

Interpolation & Exponential Search

When binary's blind halving isn't the best probe.

```
# interpolation: probe by VALUE, not position
pos = lo + (T - a[lo]) * (hi - lo) // (a[hi] - a[lo])
# uniform data: O(log log n) avg; skewed data: O(n) worst

# exponential: find range by doubling, then binary search it
bound = 1; while bound < n and a[bound] < T: bound *= 2
binary_search(a, bound//2, min(bound, n))   # O(log i): i = answer idx
```

- Interpolation = 'phone-book search': uniform keys (IDs, timestamps) converge insanely fast.
- Exponential search: unbounded/infinite arrays — no n needed, optimal when answer is near the start.
- Ternary search: two mids, for unimodal (bitonic) functions — not for sorted arrays.
- Jump search: sqrt(n) blocks — a historical curiosity; binary dominates.

STEP 6/6

Search Cheatsheet

Pick the probe for the terrain.

linear scan — O(n)

Unsorted, one query

binary search — O(log n)

Sorted array

lower/upper bound — O(log n)

Sorted + duplicates

modified binary — O(log n)

Rotated sorted

interpolation — O(log log n) avg

Uniform keys

exponential — O(log i)

Unbounded / infinite

STEP ✓I✓

Key Takeaways

Searching Algorithms — Remember

- Binary search needs sorted + random access; it halves the range each step: O(log n).
- Loop invariant 'answer in [lo,hi]' + excluding mid every step = bug-free template.
- lower_bound/upper_bound find first/last occurrences and insertion points in duplicates.
- Rotated arrays: one half is always sorted — identify it, then decide.
- Interpolation (uniform data) and exponential (unbounded) search beat blind halving.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev